

zkVM Review and Formal Verification

Ethereum Foundation cryptography internship

Yavor Litchev

10 September 2026

My internship projects

AI-assisted whitepaper review

May to July

A review pipeline, a general execution relation, and tailored W2 feedback

W1	W2	W3
Architecture	Precise specification and theorem targets	Security proofs

VanillaVM in Lean 4

July to September

Memory reconstruction, instruction semantics, segment buses, and the final theorem

The first review system

Six W1 submissions, with source evidence attached to each finding

	My prototype	Dmitry's system
Core output	Architecture JSON and review records	Detailed LaTeX feedback
Main strength	Structured checks and traceability	Precise relations and proof obligations
Initial emphasis	W1 architecture review	A roadmap for W2

I continued developing Dmitry's system for the team-facing reports

An execution relation for different zkVMs

R^* : VanillaVM's execution target

$x := (S_0, S_T)$

Public initial and final states

$w := (S_1, \dots, S_{T-1})$

Intermediate execution states

$R^*(x, w) :=$

$\forall t < T, \phi_{\text{step}}(S_t, S_{t+1})$

Program code and T are fixed

R_{exec} : an adaptable template

$x :=$ Public claim seen by the verifier

Program identity, public I/O, or boundary states

$w :=$ Program P , length T , states, instructions,
hints, and binding data a

$R_{\text{exec}}(x, w) :=$

$\text{ProgramBind}(x, P, a) \wedge \text{Init}_P(S_0, x)$

$\wedge (\forall t < T, \text{Fetch}_P(S_t) = l_t$

$\wedge \text{Step}_P(S_t, l_t, h_t, S_{t+1}))$

$\wedge \text{Term}_P(S_T, x) \wedge \text{Bind}(S_0, S_T, x)$

W2: define the team's execution relation and knowledge-soundness target

How R^* fits the template

VanillaVM's compact trace relation becomes a specialized execution relation

R^* : every adjacent pair satisfies $\varphi_{\text{step}}(S_t, S_{t+1})$

VanillaVM convention	Corresponding part of R_{exec}
Program and T are fixed	Supply them when expanding the witness
φ_{step} includes code[pc]	Derive each fetched instruction
Public claim is (S_0, S_T)	Boundary predicates enforce equality

R^* is a collapsed instance of R_{exec} under these conventions

Audit fix: make code and T explicit fixed system parameters

Testing formal versus informal content

ZisK contained both precise definitions and informal descriptions

Formal

$$C(T_1(r), \dots, T_w(r+D), a) = 0$$

Every AIR constraint, every trace row (compact notation)

Informal

“The Base Operations chips prove the base instruction set operations”

Names operations without defining semantics or constraints

Conditional

$$V_{AGG}((p, \alpha, \sigma, vk), \pi) = \text{accept}$$

Wrapper check excerpt: precise, but still needs a link to execution

Feedback and a conditional category improved the AI's classifications

Finding what the AI review missed

I hand-checked the ZisK whitepaper against its AI-generated W2 feedback

Two missed issues and two refinements to the written feedback

Hand-check finding	What needed correction
Cardinality bound	$B = 2^{20}$ followed by $\kappa < 2^B$
Leaf relation R_{LEAF}	$\text{com}(T, a)$ uses variables absent from the inputs and witness
Verifier-key roles	τ selects a child verifier, while vk carries the recursion key
Continuations and buses	Missing boundary equalities and table-bus message schemas

Hand-check → Revise the instructions → Regenerate and check the report

ZisK v2 includes all four corrections

A reusable second review pass

The checked ZisK report became an example for AI reviews of other whitepapers

1 The AI agent reads the source, rubric, and checked ZisK example

Use the team's own whitepaper and the revised report as guidance

2 Generate tailored feedback

Define an execution target and a concrete W2 roadmap

3 Audit the relations again

Check variables, bounds, public inputs, and prose against formulas

W2 roadmaps for all six submissions

ZisK, OpenVM, Succinct/SP1, Airbender, Brevis/Pico, and Lambda

Formalizing VanillaVM

Correct-trace extractability (CTE)

One extractor recovers a valid execution from every accepted proof

$$\exists E, \forall x, \pi, \text{Verify}(x, \pi) \Rightarrow \text{TraceValid}(x, E(x, \pi))$$

TraceValid requires all three

Initial state	Every transition	Final state
$\text{tr}(0) = S_0$	$\forall i < T, \text{step}(\text{tr}(i), \text{tr}(i+1))$	$\text{tr}(T) = S_T$

States contain the program counter, registers, and full memory

Probability-free model: knowledge soundness and binding are assumed perfect

My part of the Lean proof

My implementation	Role in the proof
Issue 1 Memory	Recover full states from committed states
Issue 3 Instructions	Tie each step to the operation selected by the program
Issue 5 Bus	Resolve delegated checks within each segment
Issue 7 Assembly	Use bus-checked segments in the recursive CTE theorem

The assembly builds on Jessica's recursive MultiStep layer

I also contributed shared definitions and proof-review checks

Memory reconstruction

Issue 1: reconstruct full memory from a known initial state

Full state: $S = (\text{pc}, \text{regs}, m)$

Committed state: $\hat{S} = (\text{pc}, \text{regs}, C)$

CommitInv: same pc and regs, $C = \text{Commit}(m)$

**1 Given the current invariant
and accepted step**

CommitInv $\hat{S}_1 \ S_1$
CommittedMemory.step $f \ \hat{S}_1 \ \hat{S}_2 \ w$

**2 Construct the next
full state**

Write(a, v): $m_2 := m_1[a := v]$
Read or other: $m_2 := m_1$
 $S_2 := (\hat{S}_2.\text{pc}, \hat{S}_2.\text{regs}, m_2)$

**3 Prove the next invariant
and memory semantics**

CommitInv $\hat{S}_2 \ S_2$
FullMemory.step $f \ S_1 \ S_2 \ w$

Repeat by induction to reconstruct the whole trace

f : PC/register checks. Assumptions: commitment completeness and binding.

Why update binding was needed

A commitment opening proves a value at an address, e.g. with a Merkle proof.

Position binding

Two valid openings of the same root at the same address give the same value.

Punctured binding (earlier)

A proof opening both C and C' at address a forces their valid openings elsewhere to agree.

Benedikt's counterexample

$$\text{Commit}'(m) = (\text{Commit}(m), 0)$$

Verification ignores the bit. Both earlier binding conditions still hold.

A root ending in 1 passes opening checks, but no honest commitment ends in 1.

Update binding

From $C = \text{Commit}(m)$, the same proof opens the old and new values:

$$C' = \text{Commit}(m[a := v])$$

A representative ISA

Issue 3: five operation classes with parameterized PC and register predicates

Class	Explicit memory behavior
Read	Load $m[a]$, and keep $m' = m$
Write	Set $m' = m[a := v]$
Arithmetic	Keep $m' = m$
Hash	Keep $m' = m$
Binary / bitwise	Keep $m' = m$

Five-way step $\Leftrightarrow$ semantics selected by $\text{code}[\text{pc}]$

Keeping the selected operation intact

Start with $hInv_1 : \text{CommitInv } \hat{S}_1 \ S_1$. Let $f := \text{isa.selectedMemFreePred}$.

Committed operation

$hcommitted : \text{isa.committedOperation } \hat{S}_1 \ \hat{S}_2 \ w$

Checks `code[pc]`, opening proofs, and register values

Memory reconstruction

`step_reconstruct_exact`

$hInv_2 : \text{CommitInv } \hat{S}_2 \ S_2$

$hfull : \text{FullMemory.step } f \ S_1 \ S_2 \ w$

Plain ISA step

$\text{have } hplain : \text{isa.stepPlain } S_1 \ S_2 :=$
 $\text{isa.committedOperation_stepPlain}$
 $S_1 \ S_2 \ \hat{S}_1 \ \hat{S}_2 \ w \ hInv_1 \ hInv_2 \ hcommitted \ hfull$

Concrete sanity example	Result
Correct read and a write that changes memory	Accepted
Read when the program selects write	Rejected
Write with an incorrect memory result	Rejected

One common bus within each segment

Issue 5: the segment proof checks four inner proofs under one bus commitment

Step	Keccak	Poseidon	Range
$H(B_{\text{step}}) = c$	$H(B_K) = c$	$H(B_P) = c$	$H(B_{\text{range}}) = c$

Injectivity of the bus commitment in the perfect model implies

$$B_{\text{step}} = B_K = B_P = B_{\text{range}}$$

Different segments may use different buses

Toy example: one hash request in segment 1, the same request twice in segment 2

Eliminating delegated bus checks

Use validated bus entries to recover the instruction checks delegated by the step circuit

Hash instruction

Step proof: the call is in the hash list

Hash-chip proof: all listed calls
satisfy the abstract hash predicate

⇒ This instruction's hash check holds

Binary instruction

Step proof: the inline check holds

Range-chip proof: the required
range check holds

⇒ Together they give the full
binary instruction predicate

The resulting committed ISA step preserves the memory witness w

Assembling the final CTE theorem

Issue 7: instantiate recursive leaves with the bus-checked segment system

```
theorem cte_main (h : sys.Assumptions) : sys.toZkVM.CTE := by
  change sys.toMultiStep.toZkVM.CTE
  exact sys.toMultiStep.cte (sys.multi_step_assumptions h)
```

Recursive extraction	Segment and bus proof	Memory reconstruction
Unwind convert, combine, and embed	Derive valid committed ISA steps at each leaf	Recover the full trace and exact endpoints

Leaf extractability follows from the bus theorem

A two-segment toy model checks that the assumptions can hold together

What the result establishes

Lean checks

- The assembled CTE implication
- Full-memory trace and exact endpoints
- Representative ISA and bus composition

Explicit hypotheses

Knowledge soundness, bus injectivity, and memory binding

The model still abstracts

- Concrete SNARKs and commitment schemes
- Probabilities and extractor running time
- Concrete opcodes, recursion security, and program inputs, outputs, and halting behavior

Lessons from both projects

Proofs must establish the intended VM execution

R_{exec} and the Lean definitions specify the program, valid transitions, and public claim

Human review improves the next iteration

A hand-checked report became a reusable review pass

Small counterexamples clarify assumptions

The extra commitment bit explained the gap in memory reconstruction

Thank you

Ethereum Foundation

For the opportunity to contribute

Dmitry, George, and Benedikt

For your guidance, patience, and kindness

Jessica

For your work on the Lean verification of VanillaVM

The cryptography team and the other interns

For the collaboration and conversations

Questions